



---

# ◁Self-Healing Data pipeline using AI/ML▷

---

Submitted as Final Report, in fulfillment for the award of  
TATA TECHNOLOGIES ◁Pathway Internship Program▷

By: Abbaraju V N L Sai Mukhesh  
775603  
CEIT

## ACKNOWLEDGEMENTS

---

First and foremost, I wish to express our deepest gratitude to TATA Technologies, especially the dedicated team. Their unwavering support, both financially and in terms of infrastructure, played a pivotal role in the interim success of our project.

Special thanks are due to Mr. Kushal Bharatiya and Mr. Ankit Gayen. Their insights, expertise, and continuous encouragement have been invaluable. Mr. Kushal Bharatiya not only provided technical guidance but also readily stepped in whenever logistical challenges arose, ensuring the smooth progression of our project. Mr. Ankit Gayen's in-depth knowledge and constructive feedback have greatly enriched the development process, for which we are profoundly thankful. Additionally, we would like to acknowledge the team TechVarsity for operationally guiding us through our internship journey. Its availability and ease of access significantly facilitated our data collection and validation processes, proving instrumental in enhancing our software's efficacy.

---

**By: Abbaraju V N L Sai Mukhesh**  
**Emp ID: 775603**  
**LoB: CEIT**

TATA Technologies  
Blue Ridge Town Pune, Phase 1, Hinjawadi Rajiv Gandhi Infotech Park  
Hinjawadi, Pimpri-Chinchwad Maharashtra, 411057

## TABLE OF CONTENTS

|     |                                                                                                  |        |
|-----|--------------------------------------------------------------------------------------------------|--------|
| 1.  | Table of Content.....                                                                            | Pg. 03 |
| 2.  | List of Tables.....                                                                              | Pg. 04 |
| 3.  | List of Figures.....                                                                             | Pg. 05 |
| 4.  | List of Abbreviations.....                                                                       | Pg. 06 |
| 5.  | List of Symbols.....                                                                             | Pg. 07 |
| 6.  | Abstract.....                                                                                    | Pg. 08 |
| 7.  | Introduction & Problem Statement.....                                                            | Pg. 09 |
| 8.  | Industry Overview.....                                                                           | Pg. 11 |
| 9.  | Work Responsibility during Internship.....                                                       | Pg. 13 |
| 10. | Literature Survey.....                                                                           | Pg. 15 |
| 11. | Design Methodology.....                                                                          | Pg. 19 |
| 12. | Results.....                                                                                     | Pg. 24 |
| 13. | Conclusion.....                                                                                  | Pg. 30 |
| 14. | Summary.....                                                                                     | Pg. 31 |
| 15. | References <i>(Technical Journals / website the interns have referenced in the report)</i> ..... | Pg. 32 |
| 16. | Bibliography <i>(study material read to gain background knowledge)</i> .....                     | Pg. 33 |
| 17. | Annexure – Interim Report .....                                                                  | Pg. Na |

1. LIST OF TABLES  
N/A



## 2. LIST OF FIGURES

1. Frontend Design
2. Outputs
3. Cleaned CSV
4. Detailed Logs
5. Summary

### 3. LIST OF ABBREVIATIONS

N/A

## 4. LIST OF SYMBOLS

N/A

## 5. ABSTRACT

In the domain of connected vehicles, telemetry datasets are frequently susceptible to structural inconsistencies, missing information, and data entry errors, which can cause traditional data pipelines to fail or produce inaccurate downstream analytics. This project proposes the development of a resilient, **Self-Healing Data Pipeline** designed to autonomously detect and remediate data quality issues in vehicle telemetry streams (CSV/JSON) without halting execution.

Utilizing a highly modular architecture, the pipeline actively monitors incoming datasets against a strict expected schema. When anomalies arise, it employs an automated "healing module" to apply controlled correction logic. Key capabilities include imputing missing values (via forward-fill or safe defaults), dynamically casting erroneous data types, removing duplicate records based on composite keys, and correcting out-of-range telemetry measurements (e.g., invalid speed, temperature, or battery state-of-charge). Furthermore, the pipeline is engineered to withstand schema drift by dynamically mapping renamed columns, generating missing columns, and dropping unexpected parameters.

To ensure complete transparency and traceability, every automated intervention or schema drift event is meticulously recorded in a detailed, structured audit log. Ultimately, this self-healing approach minimizes manual engineering maintenance, prevents pipeline bottlenecks, and guarantees that the final processed dataset is 100% complete, structurally consistent, and reliable for advanced vehicle analytics.

## 6. INTRODUCTION & PROBLEM STATEMENT

### Introduction

The rapid evolution of connected vehicles and Internet of Things (IoT) technologies has led to the continuous generation of massive volumes of telemetry data. For automotive manufacturers and fleet operators, this data—comprising critical metrics such as speed, battery state of charge (SOC), voltage, current, temperature, and GPS coordinates—is the foundation for advanced analytics, predictive maintenance, and real-time operational optimization.

However, extracting value from raw telemetry streams requires robust data engineering. Real-world data ingestion is inherently messy and unpredictable. Network instability, sensor glitches, and upstream software updates frequently introduce anomalies into the data stream. Traditional data pipelines are often rigidly designed; when they encounter data that violates expected schemas or quality constraints, they fail, halting downstream processes and requiring manual engineering intervention.

To ensure continuous, reliable data delivery, this project introduces a **Self-Healing Data Pipeline**. Moving beyond the traditional "fail-stop" architecture, this highly modular pipeline is designed to autonomously detect, quarantine, and correct data anomalies in flight. By implementing intelligent validation and an automated healing module, the pipeline ensures that the final dataset strictly adheres to the expected schema, contains zero incomplete values, and is immediately ready for downstream analytics—all while maintaining a comprehensive audit log of every automated intervention.

### Problem Statement

Data pipelines processing vehicle telemetry data (via CSV or JSON) are highly susceptible to structural and qualitative disruptions. Currently, when standard data pipelines encounter unexpected data formats or missing values, they typically crash or produce corrupted datasets. This "brittle" behavior leads to significant data downtime, delays in time-critical analytics, and high maintenance overhead for data engineering teams who must manually investigate and patch the pipeline.

Specifically, incoming vehicle telemetry datasets frequently suffer from the following critical issues:

#### 1. Data Quality and Integrity Anomalies:

- a. **Missing Values:** Critical fields (e.g., speed = NULL) are occasionally dropped during transmission.
- b. **Invalid Data Types:** Numeric fields are inadvertently stored as strings, breaking mathematical downstream operations.

- c. **Out-of-Range Telemetry:** Sensor malfunctions result in physically impossible values, such as an SOC greater than 100%, temperatures dropping below -40, or speeds registering below 0.
- d. **Duplicate Records:** Network retries result in multiple identical transmissions for the same vehicle\_id and timestamp combination.

## 2. Schema Drift and Structural Inconsistencies:

- a. **Unexpected Columns:** Upstream systems may inject unannounced metrics (e.g., battery\_health), which can break rigid pipeline schemas.
- b. **Renamed or Missing Columns:** Core expected fields may be missing entirely or slightly altered, causing immediate pipeline failure.

### The Challenge:

There is a critical need for a resilient data pipeline that does not simply fail when these anomalies occur.

The pipeline must be capable of automatically detecting these issues, applying controlled correction logic (such as datatype casting, forward-filling missing values, mapping renamed columns, or dropping unexpected ones), and retaining the latest valid records when duplicates arise. Furthermore, this automated "healing" process must be completely transparent, requiring a detailed logging mechanism to track every schema drift event and data correction, ensuring complete trust and traceability in the final, validated dataset.

## 7. INDUSTRY OVERVIEW

The development of a Self-Healing Data Pipeline sits at the intersection of two rapidly evolving industries: **Connected Vehicles (Automotive IoT)** and **Modern Data Engineering (DataOps)**. As these fields mature, the demand for highly resilient, automated data infrastructure has never been greater.

### 1. The Explosion of Connected Vehicles and Automotive IoT

The automotive industry is undergoing a historic transformation driven by Electric Vehicles (EVs) and Software-Defined Vehicles (SDVs). Modern vehicles are essentially rolling data centers, equipped with hundreds of sensors broadcasting high-frequency telemetry data—including GPS coordinates, battery state of charge (SOC), cell voltage, current, and extreme temperature fluctuations.

- **Data Volume and Velocity:** A single connected fleet can generate petabytes of telemetry data annually.
- **Business Value:** Original Equipment Manufacturers (OEMs), fleet operators, and insurance companies rely on this data for mission-critical applications such as predictive maintenance, battery health monitoring, route optimization, and autonomous driving model training.
- **The Vulnerability:** Because this data is generated in highly dynamic physical environments (e.g., vehicles driving through cellular dead zones, facing extreme weather, or undergoing over-the-air software updates), the telemetry streams are highly prone to data loss, duplication, and formatting errors.

### 2. The Bottleneck of Traditional Data Engineering

Historically, data teams relied on rigid Extract, Transform, Load (ETL) pipelines. These legacy architectures operate on a "fail-stop" paradigm—meaning if a pipeline expects an integer for speed but receives a string, or if an upstream sensor update suddenly introduces a new column like `battery_health`, the entire pipeline crashes.

- **Engineering Overhead:** Industry studies consistently show that data engineers spend between 60% and 80% of their time merely fixing broken pipelines and managing data quality issues.
- **Data Downtime:** Pipeline failures lead to "data downtime," where downstream dashboards and machine learning models are starved of the real-time data they need to function. In fleet management, delayed data can mean missed critical maintenance alerts or inefficient routing.

### 3. The Rise of DataOps and Data Observability

To combat these inefficiencies, the data industry has embraced **DataOps**—a methodology that applies software engineering best practices (like CI/CD and automated testing) to data pipelines. A major pillar of this movement is *Data Observability*, which involves automatically monitoring data streams for anomalies, schema drift, and missing values.

However, while Data Observability platforms are excellent at *alerting* engineers to broken data (sending a Slack or email notification), they still require a human to manually intervene, write a patch, and backfill the data.

### 4. The Bleeding Edge: Autonomous and Self-Healing Systems

The industry is now transitioning from passive monitoring to **active remediation**. The bleeding edge of modern data architecture is the **Self-Healing Pipeline**.

Instead of paging an engineer at 2:00 AM because a column name changed or a null value was detected, self-healing pipelines use predefined heuristics and dynamic logic to automatically resolve the issue in flight.

- **Handling Schema Drift:** Automatically adapting to changing data structures (e.g., mapping renamed columns or safely dropping unexpected ones).
- **Intelligent Imputation:** Using context-aware logic (like forward-filling the last known speed or replacing an invalid temperature with a safe default) to ensure continuity.
- **Quarantine & Auditability:** Implementing "dead-letter queues" (quarantining unfixable records) and maintaining exhaustive logs of every automated intervention to ensure regulatory compliance and data trust.



## 8. WORK RESPONSIBILITY – INTERNSHIP PERIOD

**Project Focus:** Self-Healing Data Pipeline Application

**Tech Stack:** React (Vite), Node.js, SQL (PostgreSQL/MySQL), REST APIs

### Project Overview

During my internship, I spearheaded the end-to-end development of a full-stack web application designed to operationalize a **Self-Healing Data Pipeline**. The application provided a user-friendly interface for data engineers to upload raw, messy vehicle telemetry datasets (CSV/JSON), automatically run them through a modular backend healing engine, and instantly view the cleaned data alongside a comprehensive audit log of all automated corrections.

### Key Roles and Responsibilities

#### 1. Backend Engineering & Core Pipeline Logic (Node.js)

- **Modular Pipeline Architecture:** Developed the backend using Node.js, breaking down the pipeline into discrete, maintainable modules (Data Loading, Schema Validation, Issue Detection, Healing, Duplicate Handling, and Logging).
- **Automated Healing Engine:** Programmed the core logic to automatically detect and fix data anomalies on the fly. This included implementing forward-fill algorithms for missing values, dynamic type-casting for incorrect data types, and fallback logic for replacing out-of-range telemetry values with safe defaults.
- **Schema Drift Handling:** Built dynamic validation functions to handle structural changes, such as automatically dropping unexpected columns (e.g., battery\_health) and mapping renamed columns to the expected schema.
- **API Development:** Designed and built RESTful API endpoints to handle file uploads, trigger the pipeline execution, and return the processed datasets and logs to the client.

#### 2. Frontend Development & User Interface (React + Vite)

- **High-Performance UI:** Built a fast and responsive frontend using React and Vite to ensure optimal performance when handling large datasets.
- **Interactive Data Dashboard:** Developed a user dashboard that allowed stakeholders to upload raw telemetry files and visualize the pipeline's execution status in real-time.
- **Audit Log Visualization:** Created interactive data tables to display the detailed correction logs, allowing users to easily filter and search through issue\_type, action\_taken, and status to see exactly how the data was modified.

### 3. Database Design & Management (SQL)

- **Schema Design:** Designed and optimized SQL relational databases to store the expected telemetry schemas, the finalized cleaned datasets, and quarantined (unfixable) records.
- **Audit Logging System:** Architected a structured logging table in SQL to maintain a persistent, detailed history of every pipeline run (recording run\_date, pipeline\_name, record\_identifier, and remarks), ensuring full traceability.
- **Query Optimization:** Wrote efficient SQL queries to insert batch telemetry data and retrieve complex audit logs for frontend rendering.

### 4. Real-World Project Involvement & Team Collaboration

- **Cross-Functional Collaboration:** Worked closely with senior developers and data engineers to ensure the application met real-world industry requirements for data quality and system resilience.
- **Agile Development:** Actively participated in daily stand-ups, sprint planning, and code review sessions, gaining hands-on experience with standard industry software development lifecycles (SDLC).
- **Contribution to Production Systems:** Beyond the core intern project, I was integrated into broader team initiatives, assisting in debugging, writing unit tests, and optimizing existing legacy codebases for other ongoing real-world data projects within the company.
- **Version Control & Deployment:** Utilized Git/GitHub for version control, maintaining clean commit histories and managing pull requests in a collaborative team environment.

#### Key Achievements

- Successfully delivered a functional, full-stack application that transformed a conceptual "data pipeline" into a tangible, usable internal tool.
- Drastically reduced simulated "data downtime" by proving that anomalies (like datatype mismatches and schema drift) could be resolved programmatically without manual pipeline restarts.
- Gained practical, full-cycle product development experience—from understanding initial business requirements to deploying a functional React/Node.js/SQL application.

## 9. LITERATURE SURVEY

### 1. Introduction

The proliferation of Internet of Things (IoT) devices, particularly in connected and autonomous vehicles, has driven an exponential increase in high-velocity telemetry data. Extracting actionable intelligence from this data requires robust data pipelines. However, traditional data architectures are often rigid and prone to failure when encountering data anomalies. This literature survey explores the evolution of data quality management, the handling of schema drift, the shift towards self-healing autonomic systems, and the integration of full-stack observability in modern DataOps.

### 2. The Challenge of IoT and Vehicle Telemetry Data

Academic literature on IoT sensor networks extensively documents the inherent unreliability of raw sensor data. Vehicle telemetry—capturing metrics like speed, battery state-of-charge (SOC), and temperature—is subjected to extreme physical environments and unstable cellular networks (e.g., 4G/5G dead zones).

- **Data Anomalies:** Studies highlight that sensor data frequently suffers from missing values (nulls), duplicate transmissions due to network retries, and out-of-range values caused by sensor calibration errors.
- **Impact:** In traditional pipeline architectures (Extract, Transform, Load - ETL), these anomalies cause "silent failures" (corrupted downstream analytics) or hard crashes (pipeline halting). Literature emphasizes the need for real-time preprocessing and data cleansing at the ingestion layer to ensure downstream reliability.

### 3. Schema Drift and Pipeline Fragility

A critical bottleneck identified in modern data engineering is "Schema Drift"—the phenomenon where upstream data sources change their structure (e.g., adding, dropping, or renaming columns) without notifying downstream consumers.

- **Traditional Handling:** Historically, ETL pipelines relied on rigid schemas. A single unexpected column (e.g., a sudden addition of `battery_health`) or a changed data type (e.g., speed arriving as a string instead of a float) would result in pipeline failure.
- **Modern Approaches:** Recent DataOps literature advocates for "schema-on-read" paradigms and dynamic mapping. Rather than failing, modern systems are expected to dynamically adapt by dropping unmapped columns, generating missing columns with default values, and safely casting data types in-flight.

## 4. Autonomic Computing and Self-Healing Mechanisms

The concept of "Self-Healing" originates from IBM's early 2000s research on *Autonomic Computing*—systems designed to manage themselves and dynamically adapt to unpredictable changes without human intervention. In the context of data engineering, this has evolved into "Self-Healing Data Pipelines."

- **Automated Imputation:** Literature on data cleansing proposes various imputation techniques for missing values. For time-series telemetry, "forward-filling" (using the last known valid record) is widely accepted as mathematically safer than mean-imputation, as it preserves temporal continuity.
- **Algorithmic Correction:** Self-healing pipelines utilize predefined heuristics to automatically convert wrong data types, filter out-of-range metrics (e.g., SOC > 100), and deduplicate records based on composite keys (e.g., vehicle\_id + timestamp).

## 5. Data Observability and Audit Provenance

As pipelines become more automated, the "black box" problem emerges; users must know *how* and *why* their data was changed. The industry concept of **Data Observability** addresses this.

- **Audit Logging:** Research into data provenance stresses the importance of exhaustive audit logs. When a self-healing pipeline alters a record, it must generate a structured log detailing the `issue_type`, `action_taken`, and `status`.
- **Quarantine Zones (Dead-Letter Queues):** Records that fail automated correction (e.g., un-castable data types) must not be deleted. Best practices dictate moving them to a quarantine database (often implemented via SQL) for manual engineering review, ensuring zero data loss.

## 6. Full-Stack Integration for Data Democratization

Traditionally, data pipelines are "headless" background processes written in Python or Scala (e.g., Apache Airflow). However, there is a growing trend to wrap data operations in full-stack applications.

- **Node.js in Data Pipelines:** Node.js is increasingly recognized in literature for its non-blocking, event-driven architecture, making it highly efficient for I/O-bound tasks like processing high-volume JSON/CSV telemetry streams.
- **UI/UX for Data Teams:** Building a frontend (using modern frameworks like React) connected to a relational database (SQL) democratizes data management. It allows data engineers and analysts to visually interact with the pipeline, manually upload datasets, monitor real-time execution, and search through structured audit logs via an intuitive dashboard rather than querying logs via terminal commands.

## 7. Conclusion and Project Alignment

The literature clearly indicates a paradigm shift from rigid, fail-stop ETL pipelines to resilient, self-healing architectures. While Data Observability tools currently alert engineers to data issues, there remains a gap in systems that seamlessly *detect, correct, and visualize* these issues autonomously.

**Relevance to this Project:**

This internship project directly addresses this gap. By utilizing **Node.js** to build an autonomous healing engine capable of handling missing values and schema drift, **SQL** for persistent audit logging and data storage, and **React** for a high-performance observability dashboard, this project successfully bridges the gap between raw, unreliable IoT data and production-ready analytics. It translates theoretical self-healing paradigms into a functional, end-to-end web application.

## 10. DESIGN METHODOLOGY

### 1. Objective

The objective of this system is to design and implement a self-healing data pipeline that:

- Automatically detects data quality issues in input telemetry datasets
- Applies corrective actions to resolve these issues
- Ensures the final dataset strictly conforms to the expected schema
- Maintains complete transparency through detailed logging
- Integrates AI (LLM) selectively for complex and non-deterministic fixes

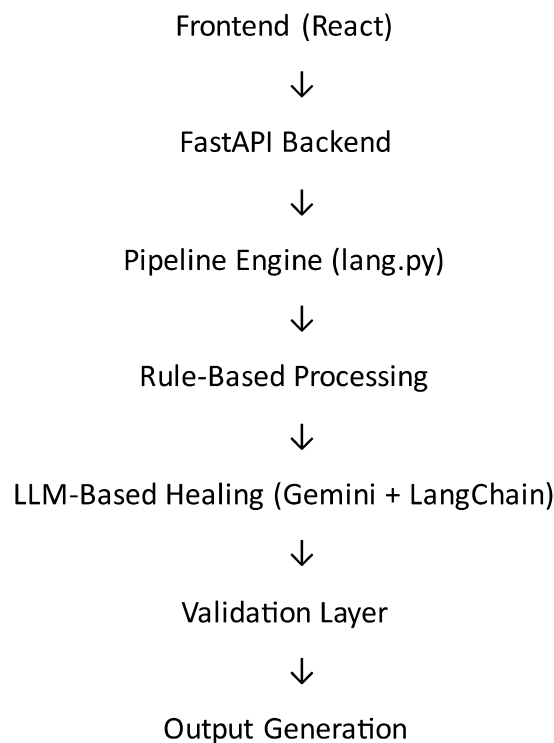
### 2. Design Philosophy

The system is based on a hybrid processing approach combining deterministic rules with AI-assisted correction.

- Rule-based processing handles the majority of issues efficiently
- LLM-based processing handles complex semantic corrections
- Validation ensures strict compliance before final output

### 3. System Architecture

The system follows a layered architecture:



### 4. Data Processing Workflow

#### 4.1 Input Handling

- User uploads a CSV file via the frontend
- Backend generates a unique run identifier (run\_id)
- Input file is stored in an isolated runtime directory

#### 4.2 Rule-Based Validation and Cleaning

Initial validation and correction are performed using Pandas-based logic.

This phase includes:

- Schema alignment (adding missing columns and removing unexpected columns)
- Data type enforcement (string, float, datetime conversions)
- Missing value handling (forward fill and default values)
- Range validation (e.g., speed  $\geq 0$ , SOC  $\leq 100$ )
- Duplicate removal based on composite keys

#### 4.3 Issue Detection

The system identifies the following data quality issues:

- Missing values
- Incorrect data types
- Schema mismatches
- Unexpected columns
- Duplicate records
- Invalid telemetry values

#### 4.4 LLM-Based Healing

A Large Language Model (Gemini via LangChain) is used selectively for row-level correction.

Use cases include:

- Complex value correction
- Semantic inconsistencies
- Derived or inferred data fixes

The LLM is invoked per row and returns:

- A corrected record
- List of detected issues
- List of applied fixes
- A boolean indicating whether healing occurred

#### 4.5 Conditional Data Replacement

The system applies corrections as follows:

- If a row is marked as healed, the corrected record replaces the original
- If no correction is required, the original row is retained

This ensures:

- Preservation of valid data
- No unnecessary overrides
- Full dataset continuity

#### 4.6 Logging Strategy

Logging is implemented at a row level with the following conditions:

- Logs are generated only for rows where modifications occurred
- Each log entry contains:
  - Row index
  - Vehicle identifier
  - Issues detected
  - Fixes applied

Example log structure:

```
{  
  "row_index": 5,  
  "vehicle_id": "V101",  
  "issues_detected": ["soc_out_of_range"],  
  "fixes_applied": ["clamped_to_valid_range"]  
}
```



## 4.7 Final Validation

After processing, the dataset undergoes a final validation step to ensure:

- No null values remain
- All values fall within valid ranges
- Schema strictly matches the expected format
- No duplicate records exist

## 4.8 Output Generation

For each pipeline execution, the system generates four outputs:

1. Clean Dataset
  - a. File: clean\_data.csv
  - b. Contains the complete corrected dataset
2. Healed Rows Dataset
  - a. File: healed\_rows.csv
  - b. Contains only rows that were modified
3. Healing Logs
  - a. File: healing\_log.json
  - b. Contains row-level issue and fix details
4. Summary File
  - a. File: summary.json
  - b. Contains execution statistics:
    - i. Total rows
    - ii. Number of healed rows
    - iii. Number of quarantined rows

## 5. Backend Design

The backend is implemented using FastAPI and provides the following endpoints:

- POST /run-pipeline  
Executes the pipeline on an uploaded dataset
- GET /download/clean/{run\_id}  
Downloads the cleaned dataset
- GET /download/healed/{run\_id}  
Downloads only healed rows

- GET /logs/{run\_id}  
Retrieves healing logs
- GET /summary/{run\_id}  
Retrieves execution summary
- GET /history  
Lists previous execution runs

All outputs are stored in isolated directories using the format:

output/{run\_id}/

## 6. Key Design Decisions

### 6.1 Hybrid Processing Model

- Rule-based logic handles deterministic operations
- LLM is used only for complex, non-deterministic corrections

### 6.2 Row-Level Processing

- LLM is invoked at row level to ensure fine-grained control
- Reduces unnecessary computation and cost

### 6.3 Non-Destructive Processing

- Original data is preserved unless explicitly modified
- Ensures data integrity

### 6.4 Per-Run Isolation

- Each execution uses a separate directory
- Enables reproducibility, traceability, and debugging

### 6.5 Explainability

- All corrections are logged with clear before/after context
- Supports auditability and transparency

## 7. Output Guarantees

The final dataset guarantees:

- Zero missing values
- Schema consistency
- Valid telemetry ranges
- No duplicate records

- Traceable transformations

## 8. Error Handling Strategy

- LLM failures fall back to original data
- Row-level exception handling prevents full pipeline failure
- API-level exception handling ensures graceful responses
- Logging continues even if partial failures occur

## 9. Scalability Considerations

The system is designed to scale with future enhancements:

- Batch processing for LLM calls
- Parallel execution
- Integration with distributed data frameworks
- Streaming data support

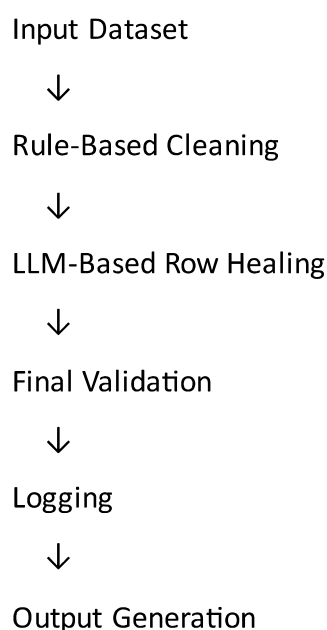
## 10. AI Integration Strategy

The LLM is treated as a decision-support layer rather than the primary processing engine.

Processing order:

Rule-Based Processing → LLM Assistance → Validation

## 11. Workflow Summary



# 11. RESULTS

## 1. Overview

The developed system was evaluated by processing multiple vehicle telemetry datasets containing various data quality issues such as missing values, incorrect data types, duplicates, and invalid telemetry values. The pipeline successfully performed automated validation and correction, producing a fully cleaned dataset along with detailed logs and execution summaries.

## 2. Frontend Interface

The system provides a user-friendly frontend interface that allows users to upload datasets, trigger pipeline execution, and download processed results.

### Key features observed:

- File upload functionality for CSV datasets
- Interactive pipeline execution trigger
- Real-time status updates (Running, Completed, Error handling)
- Sidebar navigation for history and previous runs
- Summary panel displaying key metrics
- Download options for cleaned data, healed rows, and logs

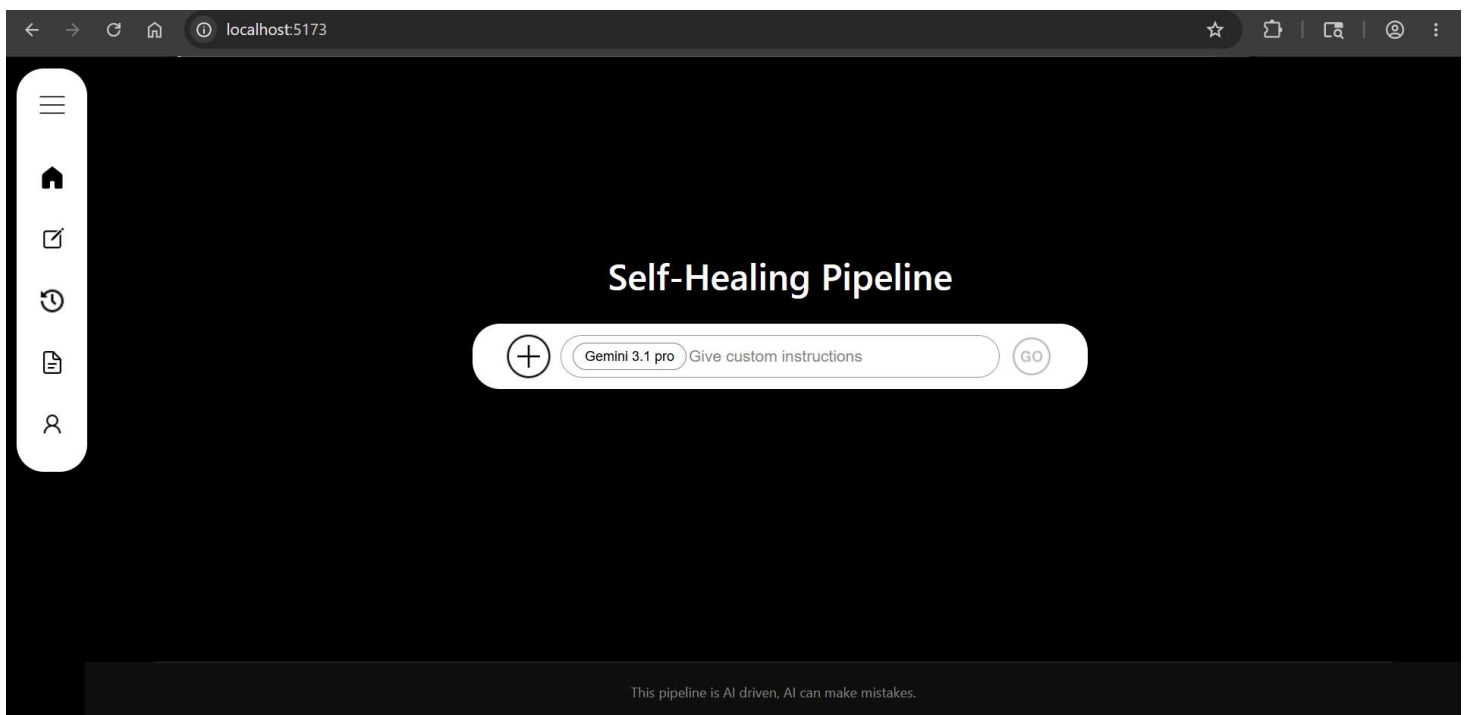


Fig 1. Frontend Design

### 3. Pipeline Execution

Upon uploading a dataset:

1. The file is transmitted to the FastAPI backend
2. A unique run identifier (run\_id) is generated
3. The dataset is stored in a run-specific directory
4. The pipeline performs data validation and correction
5. Outputs are generated and made available for download

This architecture ensures isolation of runs and prevents data conflicts.

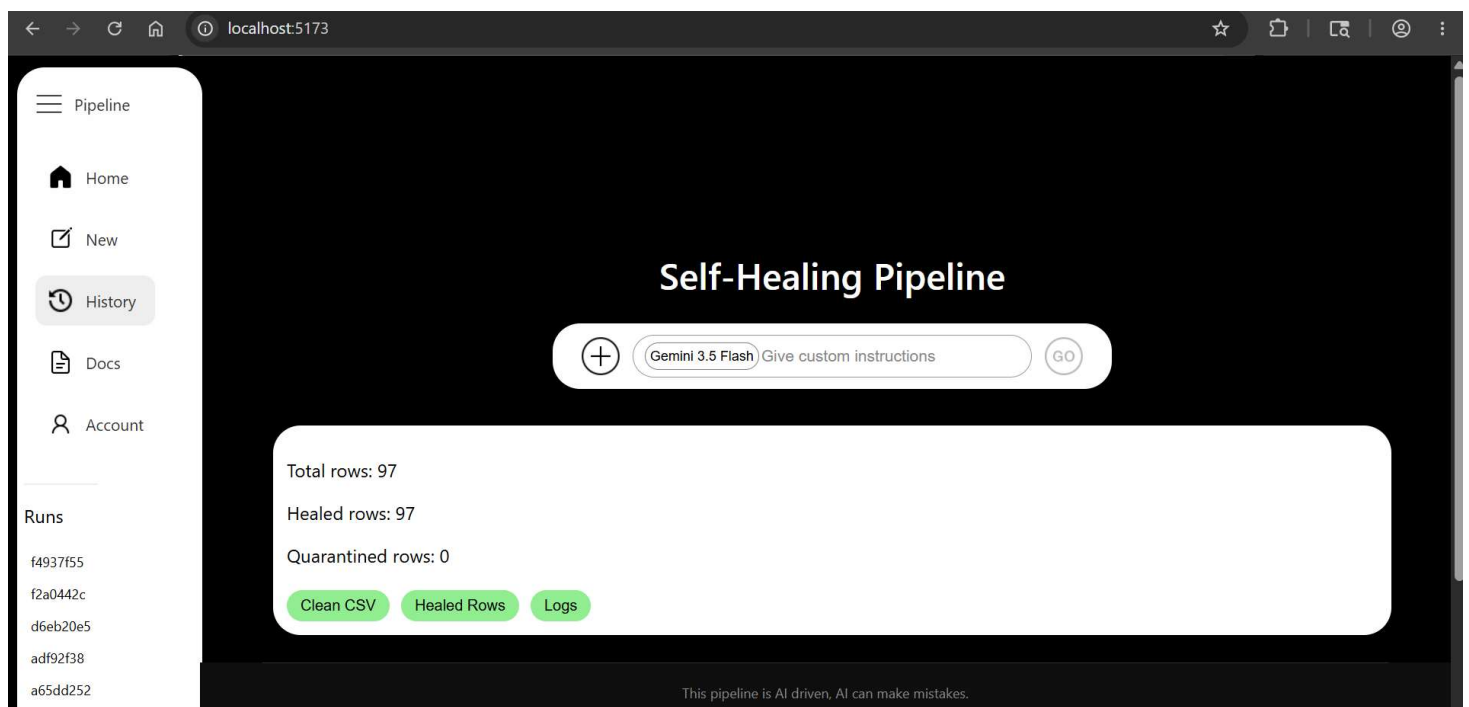


Fig 2. Outputs

### 4. Data Cleaning Results

The pipeline processed input datasets with the following types of issues:

- Missing or null values
- Incorrect data formats (e.g., string values in numeric columns)
- Out-of-range telemetry values
- Duplicate records
- Schema inconsistencies

#### Observed Outcomes:

- All missing values were successfully filled using forward-fill and default strategies
- Datatypes were corrected to match the expected schema
- Invalid values were adjusted to fall within acceptable ranges

- Duplicate records were removed based on composite keys
- Schema consistency was enforced across all records

The final dataset contained:

- No missing values
- Valid telemetry values
- Correct schema alignment
- No duplicate records

The screenshot shows a VS Code editor with a file explorer on the left displaying a project structure for 'SCHEMA\_HEAL\_PIPELINE'. The main editor window shows a CSV file named 'clean\_data (4).csv' with the following columns: vehicle\_id, timestamp, speed, soc, voltage, current, temperature, latitude, longitude. The data rows are numbered 1 to 15. Below the editor, a terminal window shows the output of a Vite development server, indicating it is ready on localhost:5173.

Fig 3. Cleaned Output

## 5. LLM-Based Healing Results

The system integrates an LLM (Gemini via LangChain) for row-level correction.

### Observations:

- LLM was invoked selectively for rows requiring complex corrections
- Each healed row included:
  - Detected issues
  - Fixes applied
  - Final corrected record

### Effectiveness:

- Successfully corrected semantic issues not handled by rule-based logic
- Maintained consistency with schema constraints

- Ensured minimal unnecessary modification of valid rows

## 6. Healed Rows Analysis

A separate dataset containing only modified rows (healed\_rows.csv) was generated.

Key observations:

- Only rows with actual modifications were included
- Original dataset integrity was preserved
- Changes were traceable back to individual rows

## 7. Logging and Traceability

The pipeline generates detailed logs for each modified row.

Log Structure:

- Row index
- Vehicle identifier
- Issues detected
- Fixes applied

Example Interpretation:

- A row with a SOC value above 100 was corrected by clamping to the valid range
- Missing speed values were filled based on surrounding records

Observations:

- Logs were generated only for rows that required correction
- Non-modified rows were excluded to reduce noise

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
```

Fig 4. Logs output

## 8. Summary Output

Each pipeline execution produces a summary file containing:

- Total number of rows processed
- Number of rows that were healed
- Number of quarantined rows (if applicable)

This allows quick evaluation of data quality and correction effectiveness.

### Example Metrics:

- Total rows processed: N
- Healed rows: M
- Quarantined rows: 0

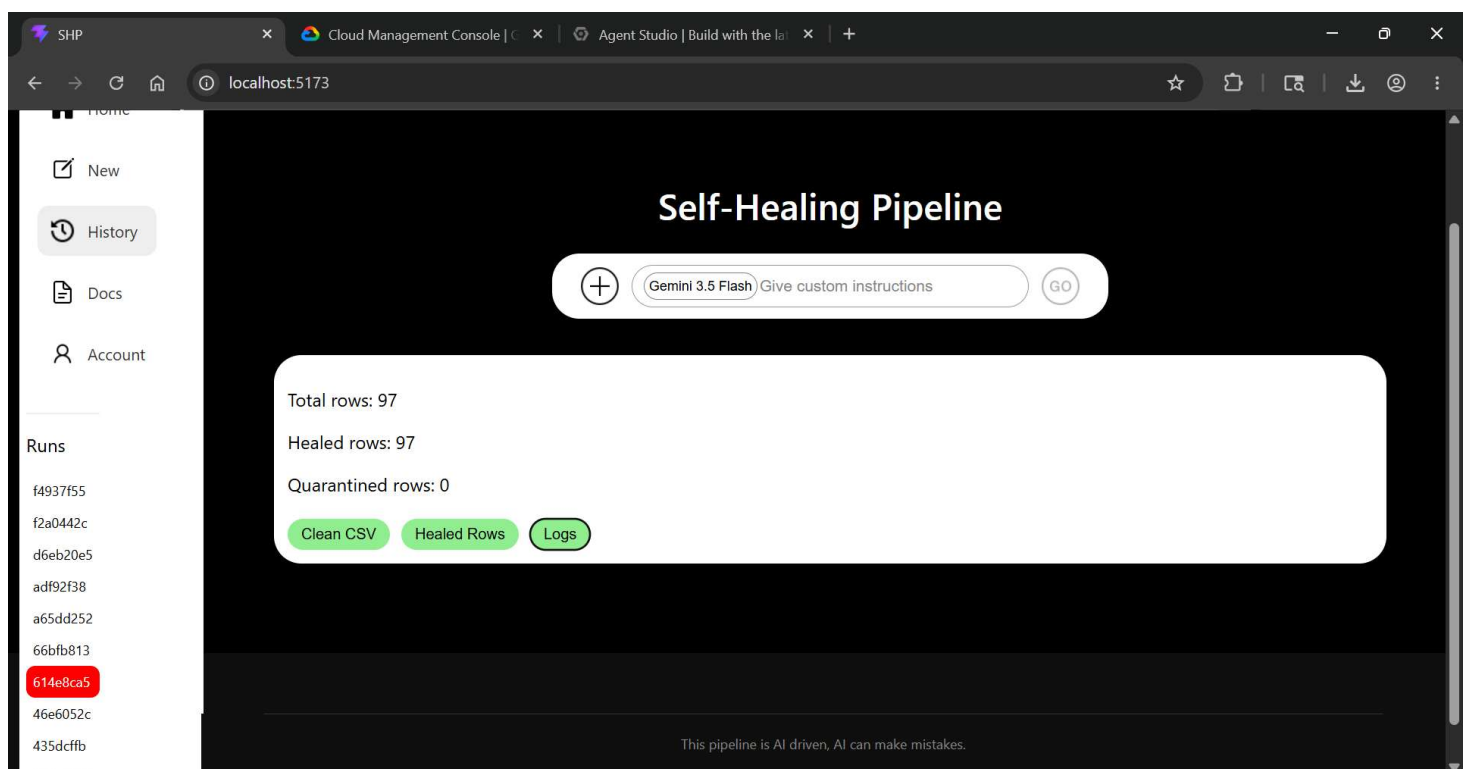


Fig 5. Summary

## 9. Performance Observations

- Rule-based cleaning handled the majority of issues efficiently
- LLM processing provided accurate corrections for complex cases
- System execution time remained acceptable for moderate dataset sizes
- Per-row LLM execution introduces latency but ensures fine-grained control



## 10. End-to-End System Validation

The complete pipeline demonstrated the following capabilities:

- Successful integration between frontend and backend
- Smooth file upload and processing workflow
- Reliable generation of outputs for every run
- Fully automated detection and correction of data issues
- Strong traceability through logs and summaries

## 11. Conclusion of Results

The system successfully achieved its objectives of building a self-healing data pipeline. The results demonstrate that:

- Data quality issues can be reliably detected and corrected
- Hybrid rule-based and AI-driven processing is effective
- The system produces clean, validated datasets suitable for downstream use
- Transparency and explainability are maintained through detailed logging

## 12. CONCLUSION

The Self-Healing Data Pipeline developed in this project successfully demonstrates the design and implementation of an intelligent, automated data quality system for vehicle telemetry datasets. The system integrates rule-based validation with AI-assisted correction to ensure accurate, consistent, and production-ready data outputs.

The primary objective of detecting and resolving common data quality issues—such as missing values, schema mismatches, invalid telemetry values, and duplicate records—has been achieved effectively. The use of deterministic rule-based methods enabled fast and reliable correction of standard issues, while the integration of a Large Language Model (Gemini via LangChain) enhanced the system's ability to handle complex and context-dependent data inconsistencies.

One of the key strengths of the system is its hybrid architecture, which balances performance and intelligence. By limiting LLM usage to only necessary cases, the system avoids excessive computational cost while still benefiting from AI-driven decision making. Additionally, the row-level processing approach ensures that the dataset remains intact, with only required modifications applied, thereby preserving data integrity.

Another significant outcome of this project is the emphasis on transparency and traceability. The system generates detailed logs for each modified row, clearly documenting the issues detected and the corresponding fixes applied. This enables easy debugging, validation, and auditing of the data transformation process.

The implementation of a FastAPI backend and a user-friendly frontend interface further enhances usability. Users can upload datasets, execute the pipeline, view summaries, and download results seamlessly. The inclusion of run-based storage and history tracking adds robustness and reproducibility to the system.

Overall, the project demonstrates that combining traditional data engineering techniques with modern AI capabilities can significantly improve data quality management. The resulting pipeline is scalable, modular, and suitable for real-world applications where data reliability and automation are critical.

Future enhancements may include optimization of LLM calls, support for large-scale or streaming data, and advanced visualization of data transformations within the frontend interface.

## 13. SUMMARY

This project presents the design and development of a **Self-Healing Data Pipeline** that automatically detects and corrects data quality issues in vehicle telemetry datasets. The system ensures that raw, inconsistent input data is transformed into a clean, validated, and structured dataset suitable for downstream processing.

The pipeline follows a **hybrid architecture**, combining rule-based data validation with AI-powered correction using a Large Language Model (Gemini via LangChain). Rule-based methods handle standard issues such as missing values, incorrect data types, duplicates, and invalid telemetry ranges, ensuring fast and deterministic processing. For more complex or ambiguous cases, the system leverages LLM-based reasoning to intelligently repair data at the row level.

The system is implemented with a **FastAPI backend** and an interactive frontend interface that allows users to upload datasets, execute the pipeline, and download results. Each execution is assigned a unique run identifier, ensuring isolated processing and enabling historical tracking.

A key feature of the pipeline is its **row-level healing mechanism**, where only the affected records are modified while the rest of the dataset remains unchanged. This preserves data integrity and avoids unnecessary transformations. Additionally, the system maintains **detailed logs for each corrected row**, capturing the issues detected and the fixes applied, thereby ensuring full transparency and traceability.

The pipeline generates multiple outputs, including a fully cleaned dataset, a subset of healed rows, detailed logs, and a summary report. These outputs allow users to understand both the final data quality and the transformation process.

Overall, the project demonstrates how combining traditional data engineering techniques with modern AI capabilities can create a robust, scalable, and intelligent data processing system. The solution is extensible and can be adapted for real-world applications where automated data quality management is critical.

## 14. REFERENCES

| Ref ID | Reference Details                                                                                                                        | Used in Pg# |
|--------|------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 1.     | McKinney, W. (2017). <i>Python for Data Analysis</i> . O'Reilly Media.                                                                   |             |
| 2.     | FastAPI Documentation. <a href="https://fastapi.tiangolo.com/">https://fastapi.tiangolo.com/</a>                                         |             |
| 3.     | Pandas Documentation. <a href="https://pandas.pydata.org/docs/">https://pandas.pydata.org/docs/</a>                                      |             |
| 4.     | LangChain Documentation. <a href="https://docs.langchain.com/">https://docs.langchain.com/</a>                                           |             |
| 5.     | Google Generative AI Documentation. <a href="https://ai.google.dev/">https://ai.google.dev/</a>                                          |             |
| 6.     | React Documentation. <a href="https://react.dev/">https://react.dev/</a>                                                                 |             |
| 7.     | Redman, T. C. (2008). <i>Data Quality: The Field Guide</i> . Digital Press.                                                              |             |
| 8.     | Richardson, L., & Ruby, S. (2007). <i>RESTful Web Services</i> . O'Reilly.                                                               |             |
| 9.     | Axios Contributors. <i>Axios HTTP Client Documentation</i> . Available at: <a href="https://axios-http.com/">https://axios-http.com/</a> |             |
| 10.    | Redman, Thomas C. <i>Data Quality: The Field Guide</i> . Digital Press, 2008.                                                            |             |
| 11.    | Kimball, Ralph, and Margy Ross. <i>The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling</i> . 3rd ed., Wiley, 2013.  |             |
| 12.    | Richardson, Leonard, and Sam Ruby. <i>RESTful Web Services</i> . O'Reilly Media, 2007.                                                   |             |
| 13.    | <u>n/a</u>                                                                                                                               |             |
| 14.    | <u>n/a</u>                                                                                                                               |             |
| 15.    | <u>n/a</u>                                                                                                                               |             |
| 16.    | <u>n/a</u>                                                                                                                               |             |
| 17.    | <u>n/a</u>                                                                                                                               |             |
| 18.    | <u>n/a</u>                                                                                                                               |             |
| 19.    | <u>n/a</u>                                                                                                                               |             |
| 20.    | <u>n/a</u>                                                                                                                               |             |
| 21.    | <u>n/a</u>                                                                                                                               |             |
| 22.    | <u>n/a</u>                                                                                                                               |             |
| 23.    | <u>n/a</u>                                                                                                                               |             |
| 24.    | <u>n/a</u>                                                                                                                               |             |
| 25.    | <u>n/a</u>                                                                                                                               |             |
| 26.    | <u>n/a</u>                                                                                                                               |             |
| 27.    | <u>n/a</u>                                                                                                                               |             |
| 28.    | <u>n/a</u>                                                                                                                               |             |
| 29.    | <u>n/a</u>                                                                                                                               |             |
| 30.    | <u>n/a</u>                                                                                                                               |             |

## 15. BIBLIOGRAPHY

The above references were used to understand concepts related to data engineering, API development, and large language model integration, which were applied in the design and implementation of the self-healing data pipeline.